



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

ارزیابی امنیتی firmware های نهفته با استفاده از ابزارهای fuzzing

اعضای گروه

علیرضا سرباز
حسین پایمرد

نام درس

آزمایشگاه اینترنت اشیا

نیم سال سوم ۱۴۰۵-۱۴۰۴

۱ چکیده

در این پروژه، فرآیند ارزیابی امنیتی فریمورهای واقعی نهفته با استفاده از ابزارهای فازینگ مخصوص سامانه‌های نهفته انجام شد. ابتدا مطالعه جامعی بر روی مقاله Scharnowski Fuzzware (al., et Security USENIX ۲۰۲۲) به عنوان یکی از پیشرفته‌ترین رویکردهای مدل‌سازی MMIO انجام گرفت و سپس ابزار P2IM روی فریمور Gateway (مبتنی بر STM32 و پروتکل Firmata) اجرا گردید. در نتیجه این اجرا، ۸۴ کرش با سیگنال SIGSEGV جمع‌آوری و تحلیل شد. تحلیل فنی یکی از کرش‌ها نشان داد که آسیب‌پذیری از نوع NULL Pointer Dereference (CWE-476) در تابع HAL_UART_TxCpltCallback است که با یافته‌های مقاله P2IM (CWE-787 + CWE-129) هم‌خانواده محسوب می‌شود.

۲ مقدمه و انگیزه

دستگاه‌های embedded به دلیل گستردگی و ارزش بالا هدف جذابی برای مهاجمان‌اند، اما fuzzing آن‌ها به‌طور مستقیم روی سخت‌افزار به دلیل منابع محدود و کندی اجرا عملی نیست. راه‌حل رایج re-hosting است: اجرای firmware در یک محیط شبیه‌سازی‌شده به جای سخت‌افزار واقعی.

چالش اصلی re-hosting برای firmware‌های مونولیتیک (باینری خام بدون متادیتای سیستم‌عامل) این است که باید تعامل firmware با سخت‌افزار از طریق MMIO (ورودی/خروجی نگاشت‌شده در حافظه) به درستی شبیه‌سازی شود. روش‌های قبلی یا نیاز به تلاش دستی زیاد برای مدل‌سازی هر peripheral دارند (مثل QEMU) یا از مدل‌های ایستا و تقریبی استفاده می‌کنند که دقت کافی ندارند. راه‌حل ساده‌تر این است که مقادیر MMIO را مستقیماً توسط fuzzer تولید کرد، اما این کار باعث می‌شود fuzzer باید حجم عظیمی از بیت‌های بی‌ربط (اصطلاحاً input overhead) را حدس بزند، که کارایی fuzzing را به شدت پایین می‌آورد.

۳ مرور مقاله Fuzzware

۱-۳ مفهوم Input Overhead

نویسندگان دو نوع overhead تعریف می‌کنند:

- Full input overhead: هیچ بیتی از مقدار تولیدشده توسط fuzzer روی منطق firmware اثر ندارد (مثلاً یک رجیستر پیکربندی که فقط نوشته می‌شود و خوانده نمی‌شود).

- Partial input overhead: فقط بخشی از بیت‌ها معنادارند؛ مثلاً یک رجیستر ۳۲-بیتی خوانده می‌شود اما فقط ۸ بیت آن در تصمیم‌گیری کد استفاده می‌شود.

در مثال‌های واقعی مقاله (مثل تابع دریافت کاراکتر از پورت سریال)، تا ۹۲٪ از بایت‌های خوانده‌شده از MMIO عملاً هیچ تأثیری روی رفتار برنامه ندارند.

۲-۳ رویکردهای پیشین و محدودیت‌های آن‌ها

- High-level emulation: به جای شبیه‌سازی MMIO، کتابخانه‌های سطح بالا هوک می‌شوند؛ نیاز به شناخت دقیق کتابخانه‌های هدف دارد.

- Pattern-based modeling (مثل P2IM): با تشخیص الگوی دسترسی به رجیستر، یک مدل از پیش تعریف‌شده به آن نسبت داده می‌شود؛ اما چون به منطق داخلی firmware نگاه نمی‌کند، نمی‌تواند partial overhead را حذف کند.

- Guided symbolic execution (مثل µEmu، Laelaps): رجیسترهای سخت‌افزاری را به صورت نمادین (symbolic) در نظر می‌گیرند و مقدار را در جهت مسیرهای امیدبخش حل می‌کنند؛ اما این کار به قیمت حذف برخی مسیرهای اجرایی (path elimination) تمام می‌شود که می‌تواند بخشی از رفتار firmware (از جمله مسیرهای خطا که غالباً محل باگ‌اند) را از دسترس fuzzer خارج کند.

نویسندگان سه مشکل مشترک را در همه رویکردهای قبلی شناسایی می‌کنند: نیاز به تلاش دستی برای هر firmware، حذف ناقص overhead، و حذف مسیرهای اجرایی (path elimination).

۳-۳ طراحی Fuzzware

ایده اصلی Fuzzware این است که برای هر دسترسی منحصر به فرد به MMIO (ترکیب آدرس حافظه و program counter)، یک‌بار از dynamic symbolic execution محلی (محدود به بازه کوتاهی از اجرا، نه کل برنامه) استفاده شود تا مشخص شود دقیقاً کدام بخش از مقدار سخت‌افزاری روی

منطق firmware اثر می‌گذارد. نتیجه یک «مدل دسترسی» (access model) است که تعیین می‌کند fuzzer باید چند بیت ورودی برای این دسترسی خرج کند و آن بیت‌ها چگونه به مقدار نهایی تبدیل شوند.

نکته مهم: این مدل‌ها هیچ تصمیمی درباره اینکه کدام مسیر بهتر است نمی‌گیرند؛ فقط مجموعه گزینه‌های اصلی را با کمترین overhead در اختیار fuzzer می‌گذارند. بنابراین برخلاف µEmu/Laelaps، هیچ مسیر اجرایی حذف نمی‌شود.

معماری کلی شامل یک emulator ISA (مبتنی بر Unicorn Engine) است که با یک موتور fuzzing پوشش‌محور (AFL/AFL++) جفت شده. هر بار که آدرس دسترسی MMIO جدیدی دیده شود، به صورت موازی یک snapshot از وضعیت رجیسترها و حافظه گرفته می‌شود و به موتور DSE (بر پایه angr) داده می‌شود تا مدل مناسب استخراج شود؛ سپس این مدل به صورت پویا به پیکربندی emulator اضافه می‌شود.

۴-۳ پنج مدل دسترسی MMIO

- Constant (نوع overhead: کامل): مقدار باید دقیقاً برابر یک عدد ثابت خاص باشد تا اجرا ادامه یابد (مثل پرچم status). پارامتر: مقدار ثابت. مثال: انتظار برای HAS_DATA در حلقه polling.
- Passthrough (نوع overhead: کامل): مقدار هیچ اثری روی وضعیت برنامه ندارد و مثل حافظه معمولی رفتار می‌کند. پارامتر: —. مثال: خواندن رجیستر GPIO قبل از نوشتن یک بیت.
- Bitextract (نوع overhead: جزئی): فقط بخشی از بیت‌ها با یک bitmask استخراج و استفاده می‌شود. پارامتر: bitmask. مثال: خواندن ۴ بایت داده سریال، استفاده فقط از ۱ بایت.
- Set (نوع overhead: جزئی): مقدار بین چند گزینه گسسته انتخاب می‌شود که هر کدام مسیر متفاوتی را فعال می‌کند. پارامتر: لیست مقادیر. مثال: انتخاب بین ۴ عملیات در یک switch/case.
- Identity (نوع overhead: هیچ): حالت پیش فرض محافظه کارانه؛ همه بیت‌ها معنادار فرض می‌شوند یا DSE به نتیجه نرسیده. پارامتر: —. مثال: متغیر نمادین از محدوده تحلیل خارج

شده.

اگر چند مدل همزمان قابل اعمال باشند، مدلی انتخاب می‌شود که بیشترین کاهش overhead را ایجاد کند.

۳-۵ محدوده تحلیل (Modeling Scope)

برای جلوگیری از انفجار حالت (state explosion) در symbolic execution، تحلیل فقط تا رسیدن به یکی از این شرایط ادامه می‌یابد: تمام متغیرهای نمادین ردیابی‌شده «مرده» شوند (دیگر در حافظه/رجیستر استفاده نشوند)، تابع فعلی بازگردد، متغیر نمادین از محدوده تحلیل خارج شود (مثلاً در حافظه سراسری نوشته شود)، یا محدودیت منابع محاسباتی (timeout یا تعداد گام‌ها) تمام شود. این محدودسازی بر این فرض استوار است که وضعیت رجیسترهای MMIO کوتاه‌عمر است و firmware معمولاً بلافاصله بعد از خواندن، از مقدار استفاده کرده یا آن را دور می‌ریزد.

۳-۶ پیاده‌سازی

پیاده‌سازی مرجع Fuzzware برای معماری ARM Cortex-M ساخته شده (به دلیل رواج بالا در صنعت). از Unicorn Engine به‌عنوان emulator ISA، AFL، کلاسیک و AFL++ به‌عنوان موتور fuzzing، و angr به‌عنوان موتور DSE استفاده شده است. برای مدیریت تایمرها و وقفه‌ها، کنترلر وقفه (NVIC) و تایمر SysTick استاندارد Cortex-M در Unicorn پیاده‌سازی شده تا اجرا کاملاً قابل بازتولید (deterministic) باشد؛ این ویژگی برای تولید snapshot‌های سازگار در طول مدل‌سازی موازی ضروری است.

زمان پیش‌فرض برای هر عملیات DSE، ۵ دقیقه و حداکثر ۱۰۰۰ بلوک پایه است. مصرف ورودی خام نیز در سطح بایت انجام می‌شود (نه بیت)، چون سازگاری بهتری با ابزارهای fuzzing استاندارد دارد.

۳-۷ نتایج ارزیابی

ارزیابی روی ۷۷ نمونه firmware از ۱۹ پلتفرم سخت‌افزاری مختلف انجام شده است:

- کاهش فضای ورودی: بین ۳.۴۹٪ تا ۴.۸۳٪ در پیاده‌سازی سطح بایت، و تا ۵.۹۵٪ در حالت فرضی سطح بیت.
- هزینه مدل‌سازی: به‌طور میانگین ۶۲ مدل در هر آزمایش ۲۴ ساعته تولید شده که هرکدام حدود ۶ ثانیه طول کشیده (کمتر از ۲٪ زمان کل آزمایش).
- پاس شدن تمام ۶۶ تست واحد P2IM؛ Fuzzware اولین سیستم خودکار عمومی است که به این نتیجه رسیده (روش‌های قبلی ۸۳٪ و ۹۵٪ را پاس کرده بودند).
- در مقایسه مستقیم با P2IM و μ Emu روی ۲۱ نمونه firmware واقعی، Fuzzware به‌طور میانگین حدود ۴۴٪ پوشش کد بیشتر از P2IM و ۶۱٪ بیشتر از μ Emu کسب کرده؛ در برخی موارد (مثل نمونه CNC) پوشش دو تا سه برابر شده است.
- در ۱۹ از ۲۱ مورد، حتی بدترین اجرای Fuzzware از بهترین اجرای هر دو روش رقیب بهتر بوده است.
- کشف ۳ باگ گزارش‌نشده در نمونه‌های استفاده‌شده در کارهای رقیب (Soldering Iron، CNC، GPSTracker).
- در آزمایش روی پشته‌های شبکه Zephyr و Contiki-NG، مجموعاً ۱۵ آسیب‌پذیری ناشناخته کشف و ۱۲ CVE برای آن‌ها ثبت شده است.

۸-۳ تحلیل علت اصلی کرش‌ها

از میان ۶۱ کرش یکتای تولیدشده در کل آزمایش‌ها:

- ۴۲ مورد آسیب‌پذیری امنیتی واقعی بودند.
- ۱۶ مورد ناشی از عدم بررسی صحیح مقداردهی اولیه (initialization) بودند.
- فقط ۳ مورد false positive تشخیص داده شدند (یعنی باگ در کد وجود دارد اما روی سخت‌افزار واقعی به دلیل محدودیت‌های فیزیکی رخ نمی‌دهد؛ مثلاً یک فیلد رجیستر که firmware فرض کرده حداکثر مقدار ۱۵ دارد اما اجازه داده هر مقدار ۸ بیتی در آن قرار گیرد).

۹-۳ محدودیت‌ها و بحث

- DMA به‌طور کامل و خودکار مدل‌سازی نمی‌شود؛ فقط از طریق پیکربندی دستی بافرهای DMA به‌عنوان ناحیه MMIO پشتیبانی جزئی می‌شود.
- مدل‌های تولیدشده به قالبی وابسته‌اند که فقط به آدرس، اندازه دسترسی و program counter نیاز دارد؛ به همین دلیل نویسندگان نشان داده‌اند این مدل‌ها می‌توانند مستقل از Fuzzware نیز در فریم‌ورک‌های دیگر (مثل avatar2) استفاده شوند.
- در موارد نادر (کمتر از ۵.۲٪ مدل‌ها)، وقتی DSE به محدودیت منابع می‌رسد، به‌صورت محافظه‌کارانه مدل Identity اعمال می‌شود تا هیچ مسیری اشتباهاً حذف نشود.

۴ اجرای P2IM روی فریمور Gateway

۱-۴ آماده‌سازی محیط

فریمور Gateway (مبتنی بر STM32 و پروتکل Firmata) به‌عنوان هدف انتخاب شد. محیط اجرا با P2IM آماده‌سازی و فریمور در QEMU اجرا شد.

۲-۴ اجرای فازینگ

کمپین فازینگ با P2IM اجرا شد و ۸۴ کرش با سیگنال SIGSEGV (sig:11) جمع‌آوری گردید.

۳-۴ Triage و کاهش کرش‌ها

کرش‌ها با استفاده از replay در QEMU و استخراج PC و backtrace تحلیل شدند. آدرس‌های کرش با arm-none-eabi-addr2line و objdump به کد منبع نگاشت شدند.

۵ تحلیل فنی کرش انتخابی

۱-۵ نقطه دقیق کرش

```
08008774 <HAL_UART_TxCpltCallback>:
8008776: bl 800872c <uart_index> ; r0 = uart_index(huart)
800877a: ldr r3, [pc, #56] ; r3 = &table_A (0x200006a8)
800877c: ldr.w r4, [r3, r0, lsl #2] ; r4 = table_A[r0] (huart handle)
8008780: cmp r0, #4
8008782: bls.n 8008786 ; if r0<=4 continue
8008784: pop {r4, pc} ; else return
8008786: ldr r3, [pc, #48] ; r3 = &table_B (0x20000694)
8008788: ldr.w r3, [r3, r0, lsl #2] ; r3 = table_B[r0] (callback fn ptr)
800878c: mov r0, r4
800878e: blx r3 ; <<<<< crash point
```

۲-۵ مسیر اجرا

۱. ورودی فازینگ از طریق Data Register (DR) به عنوان بایت‌های سریال به فریمور ارسال می‌شود.

۲. این ورودی باعث trigger شدن یک UART TX-Complete interrupt می‌شود.

۳. زنجیره اجرا: USARTx_IRQHandler ← HAL_UART_IRQHandler ← HAL_UART_TxCpltCallback

۴. در HAL_UART_TxCpltCallback، کد از جدول table_B یک function pointer می‌خواند.

۵. مقدار خوانده‌شده صفر (NULL) است، اما کد بدون بررسی null بودن، دستور blx r3 را اجرا می‌کند.

۶. نتیجه: پرش به آدرس 0x00000000 ← خطای UC_ERR_FETCH_UNMAPPED ← SIGSEGV.

۳-۵ بخش آسیب‌پذیر

- تابع: HAL_UART_TxCpltCallback
- آدرس: 0x8008774
- فایل: uart.c (پیاده‌سازی Arduino_Core_STM32)
- نوع خطا: Call through unvalidated/NULL function pointer

۴-۵ علت ریشه‌ای

- تابع uart_index() شماره کانال UART فعال (۰ تا ۴) را برمی‌گرداند.
- کد از این ایندکس برای خواندن یک function pointer از جدول table_B در آدرس 0x20000694 استفاده می‌کند.
- هیچ بررسی‌ای برای null بودن مقدار خوانده‌شده انجام نمی‌شود.
- در این کرش، r3 = 0x00000000 بود — یعنی برای این کانال UART، هیچ callback ای register نشده بود.
- کد بدون چک، سعی کرد به آدرس 0x0 جامپ کند.

۵-۵ دسته‌بندی CWE

- NULL Pointer Dereference :CWE-476
- Unchecked Return Value (الگوی Missing Validation before use) :CWE-252

۶ تطبیق با مقاله P2IM

۱-۶ باگ گزارش شده در مقاله

طبق مقاله P2IM (بخش ۴.۵): ترکیب CWE-129 (Improper Validation of Array Index) + CWE-787 (Out-of-Bounds Write). این باگ به مهاجم remote اجازه می‌دهد اشیاء داده را روی دستگاه embedded بازنویسی کند و باعث denial of service یا خرابی داده شود.

۲-۶ مقایسه یافته ما با مقاله

- CWE: مقاله P2IM دارای CWE-787 + CWE-129 است؛ یافته ما -CWE-476 (252 است).

- مکانیزم: در مقاله، عدم اعتبارسنجی ایندکس آرایه منجر به نوشتن خارج از محدوده می‌شود؛ در یافته ما، عدم بررسی NULL بودن function pointer منجر به پرش به آدرس 0x0 می‌شود.

آسیب: در مقاله، Firmata parser (احتمالاً sysex handler) است؛ در یافته ما، HAL_UART_TxCpltCallback، در uart.c است.

- نوع تأثیر: در مقاله، بازنویسی داده / DoS است؛ در یافته ما، DoS (Crash) است.

۳-۶ تحلیل تطبیق

اگرچه CWE دقیق یافته ما با مقاله متفاوت است، اما هم خانواده محسوب می‌شود:

- هر دو ناشی از عدم اعتبارسنجی داده‌ای هستند که از مسیر ارتباطی ورودی (Firmata/سریال) می‌آید.

- هر دو منجر به استفاده از یک مقدار حساس (آرایه یا function pointer) بدون بررسی صحت می‌شوند.

- طبق شرح پروژه، وجود CWE/CVE یکسان الزامی نیست و کافی است نشان دهیم یافته از همان کلاس مسائلی است که مقاله در بخش Bug Case Studies توصیف کرده است.

۷ نکته احتیاطی: احتمال False Positive

مقاله P2IM (بخش ۳.۳ "Infeasible Peripheral Inputs & False Positives") هشدار می‌دهد که چون فریمور با تایمینگ interrupt دلخواه فازر اجرا می‌شود، ممکن است سناریویی پیش بیاید که در سخت‌افزار واقعی هرگز رخ ندهد.

سوال کلیدی: آیا در سخت‌افزار واقعی ممکن است یک TX-Complete interrupt برای کانالی که هیچ callback ثبت نکرده، fire شود؟

بررسی لازم: باید initialization code فریمور Gateway بررسی شود تا مشخص گردد:

- آیا مسیری وجود دارد که UART فعال شود ولی callback ثبت نشود؟
- اگر چنین مسیری reachable نباشد، این یافته می‌تواند یک false positive ناشی از مدل P2IM interrupt-firing باشد.

نکته مشابه در Fuzzware: مقاله Fuzzware نیز اشاره می‌کند که ۳ مورد از ۶۱ کرش یکتا false positive بودند — مثلاً یک فیلد رجیستر که فریمور فرض کرده حداکثر مقدار ۱۵ دارد اما اجازه داده هر مقدار ۸ بیتی در آن قرار گیرد.

۸ جمع‌بندی و مستندسازی نهایی

- id:000000, sig:11, src:000019+000059, op:splice, rep:32 :Crash input
- مسیر اجرا: ورودی از DR ← UART TX-complete interrupt ← USARTx_IRQHandler ← HAL_UART_TxCpltCallback ← HAL_UART_IRQHandler
- بخش آسیب‌پذیر: HAL_UART_TxCpltCallback @ 0x8008774 در uart.c
- نحوه رخداد: blx r3 با r3=0x0 ← UC_ERR_FETCH_UNMAPPED
- علت ریشه‌ای: فقدان null-check روی function pointer خوانده‌شده از جدول callback قبل از فراخوانی غیرمستقیم
- CWE: CWE-476 (Null Pointer Dereference) + CWE-252 (Unchecked Return Value)

- تطبیق با مقاله: هم‌خانواده با CWE-787+CWE-129 مقاله (عدم اعتبارسنجی داده ورودی در آدرس‌دهی حساس)
- هشدار: احتمال false positive به دلیل مدل interrupt-firing P2IM — نیازمند بررسی initialization code

۹ مقایسه P2IM و Fuzzware

- رویکرد مدل‌سازی: Fuzzware از DSE محلی + پنج مدل دسترسی استفاده می‌کند؛ P2IM از Pattern-based modeling.
- حذف Partial Overhead: Fuzzware: بله (Bitextract, Set)؛ P2IM: خیر.
- Path Elimination: Fuzzware ندارد (همه مسیرها حفظ می‌شوند)؛ P2IM ندارد (اما مدل‌های ناقص).
- نیاز به تلاش دستی: Fuzzware کم (خودکار)؛ P2IM متوسط (نیاز به تعریف الگوها).
- پوشش کد: Fuzzware بالاتر (۴۴٪ بیشتر از P2IM)؛ P2IM پایه.
- معماری پشتیبانی‌شده: Fuzzware دارای ARM Cortex-M؛ P2IM دارای ARM Cortex-M.
- Emulator: Fuzzware دارای Unicorn Engine؛ P2IM دارای QEMU.
- Fuzzing Engine: Fuzzware دارای AFL/AFL++؛ P2IM دارای AFL.
- DSE Engine: Fuzzware دارای angr؛ P2IM ندارد.
- False Positive: Fuzzware ۳ از ۶۱؛ P2IM گزارش شده در بخش ۳.۳.

۱۰ محدودیت‌ها

۱-۱۰ محدودیت‌های Fuzzware

- DMA به‌طور کامل و خودکار مدل‌سازی نمی‌شود (فقط پیکربندی دستی).

- در موارد نادر ($> 5.2\%$ مدل‌ها)، وقتی DSE به محدودیت منابع می‌رسد، مدل Identity اعمال می‌شود.
- فقط از ARM Cortex-M پشتیبانی می‌کند.

۱۰-۲ محدودیت‌های P2IM

- نمی‌تواند partial input overhead را حذف کند.
- مدل‌های pattern-based ممکن است برخی رجیسترها را اشتباه دسته‌بندی کنند.
- احتمال false positive به دلیل مدل interrupt-firing دلخواه.

۱۱ ادامه کار

۱. بررسی ۸۴ کرش باقی‌مانده: تعیین اینکه چند مورد به همان `blx r3 @ 0x800878e` می‌رسند (یعنی همه یک باگ واحد هستند).
۲. بررسی initialization code فریمور Gateway: برای رد یا تایید false positive بودن یافته.
۳. تطبیق دقیق‌تر با مقاله P2IM: جستجوی کرش‌هایی که دقیقاً به `CWE-787+CWE-129` گزارش شده در مقاله برسند.
۴. استفاده از مدل‌های Fuzzware در ابزارهای دیگر: طبق مقاله، مدل‌های تولید شده می‌توانند مستقل از Fuzzware در فریمورک‌هایی مثل `avatar2` استفاده شوند.
۵. اجرای Fuzzware روی اهداف جدید: با توجه به این‌که Fuzzware پوشش کد بالاتری نسبت به P2IM دارد، می‌توان همان فریمور Gateway را با Fuzzware نیز فاز کرد و نتایج را مقایسه کرد.

۱۲ نتیجه‌گیری

در این پروژه، دو رویکرد مکمل در حوزه فازینگ فریمورهای نهفته بررسی شد:

- Fuzzware با مدل‌سازی دقیق MMIO از طریق DSE محلی، توانست input overhead را تا ۵۰۹۵٪ کاهش دهد و ۱۲ CVE کشف کند. این ابزار اولین سیستم خودکار عمومی است که ۱۰۰٪ تست‌های واحد P2IM را پاس می‌کند.

- P2IM به‌عنوان یک رویکرد pattern-based، روی فریمور Gateway اجرا شد و ۸۴ کرش تولید کرد. تحلیل فنی یکی از کرش‌ها منجر به کشف یک NULL Pointer Dereference (CWE-476) در HAL_UART_TxCpltCallback شد که از نظر کلاس آسیب‌پذیری با باگ گزارش شده در مقاله P2IM (CWE-129 + CWE-787) هم‌خانواده است.

این نتایج نشان می‌دهد که ترکیب رویکردهای مدل‌سازی دقیق (Fuzzware) با ابزارهای pattern-based (P2IM) می‌تواند تصویر جامع‌تری از آسیب‌پذیری‌های فریمورهای نهفته ارائه دهد. همچنین، توجه به احتمال false positive — به‌ویژه در سناریوهای interrupt-firing — برای اعتبارسنجی یافته‌ها ضروری است.